E-COMMERCE REVENUE & CUSTOMER RETENTION ANALYTICS

COMPLETE PROJECT DOCUMENTATION

Everything in one place: what the project is, how to install and run it, how every part works, what the finished dashboard contains, how to finish and publish it, and how to fix common problems.

Field	Detail
GitHub	https://github.com/MARCUS-00/ecommerce-retention-analytics
Dataset	Brazilian E-Commerce Public Dataset by Olist (Kaggle) — ~100,000 orders, 2016-2018
Stack	PostgreSQL 16 (Docker) · Python 3.12 · SQL · Jupyter · Power BI Desktop
Setup time	~30 minutes, first run
Cost	₹0 — every tool used is free

About This Document

This is the single, complete reference for the project. It is based on the repository (README.md, docs/PLAYBOOK.md, docs/findings.md, the dashboard folder, and the full src/, sql/, notebooks/ and tests/ trees) and on the dashboard checks recorded in dashboard/BUILD_QA_REPORT.md.

Table of Contents

How to Use This Document **4**

1. What This Project Is **5**

1.1 Why This Project Is Well-Built 5

1.2 The Dataset and Its Three Traps 5

1.3 Headline Findings 6

1.4 What the Findings Mean for the Business 6

2. Architecture & How Data Flows **7**

2.1 The Two Database Areas 8

2.2 Two Populations That Are Never Mixed 8

2.3 Two Environments 8

3. Technology Stack **8**

3.1 Python Libraries Used 9

4. What Exists — and What Does Not **9**

5. Prerequisites **10**

5.1 Hardware 10

5.2 Software You Must Install Yourself 10

5.3 Verify Your Prerequisites 11

6. Get the Project Code **11**

7. Install make **11**

7.1 Recommended: Scoop (no admin rights needed) 11

7.2 Alternative: Chocolatey (needs an elevated shell) 11

8. Set Up Kaggle Credentials **12**

9. Configure the Project **12**

9.1 Environment Variables (.env) 12

9.2 docker-compose.yml, Explained 13

10. Start the Database & Install Dependencies **13**

11. Run the Pipeline **13**

12. Run the Tests **14**

13. Project Structure **15**

14. Database & SQL Reference **15**

14.1 Fact Tables 16

14.2 Dimension Tables 16

14.3 SQL Files by Folder 17

14.4 The 18 Analysis Queries, by Stakeholder 17

14.5 Example Query — Repeat-Purchase Rate 18

14.6 Verifying the Database Directly 18

14.7 Inspecting the Database Directly (Troubleshooting Tool) 18

15. Python Code Reference **18**

15.1 Validation Checks in Detail 19

15.2 Verified Code Quality....19
16. Docker Reference....20
17. Developer Guide....21
17.1 How to Add a New Analysis Query....21
17.2 Testing & CI....21
18. Notebook Guide....22
18.1 Notebook 01 — Exploratory Data Analysis....22
18.2 Notebook 02 — Retention Cohorts & RFM....22
18.3 Notebook 03 — Delivery vs. Reviews (Statistical Test)....22
18.4 Notebook 04 — 28-Day Demand Forecast....23
18.5 Confirming Real Data Was Used....23
19. Power BI Desktop & DAX Guide....24
19.1 Install Power BI Desktop....24
19.2 Connect to the Database....24
19.3 The Authentication Fix Already in Place....24
19.4 Tables in the Final Model....25
19.5 Model Relationships....25
19.6 DAX Measures....25
19.7 The Four Dashboard Pages....27
19.8 Refresh, Save and Screenshots....28
19.9 Known Limits....28
20. The Insights Memo....29
20.1 Source Findings....29
20.2 Structure of the Memo....29
21. Before Your Final Push....30
22. Troubleshooting....31
23. Best Practices & Maintenance....32
24. Frequently Asked Questions....32
25. Verifying Your Results....33
25.1 Independent Re-run, October 2026....33
26. Final Verification Checklist....33
27. Glossary....34
28. Appendix: Project at a Glance....35
28.1 Quick Command Reference....35

How to Use This Document

This guide covers installing, running, understanding, finishing and troubleshooting the project in one file. It follows the order in which you would work through it:

Part	Covers	Read this if...
I — Overview	What the project is, its architecture, its tech stack	You're new to the project
II — Installation & Setup	Every command from an empty machine to a running pipeline	You're setting it up for the first time
III — Understanding the Code	Folder structure, database, SQL, Python, Docker, dev guide	You want to read, modify, or explain the code
IV — Running the Analysis	The four Jupyter notebooks in depth	You want to understand or re-run the analysis
V — The Dashboard	What the finished dashboard contains, how it connects to the database, and how to refresh or rebuild it	You want to open, refresh or rebuild the dashboard
VI — Finishing the Project	The insights memo and pre-push housekeeping	You're close to done
VII — Operating the Project	Troubleshooting, best practices, FAQ	Something isn't working, or you want to do it right
VIII — Reference	Findings, checklist, glossary, appendix	You need to look something up fast

PART I — OVERVIEW

1. What This Project Is

Imagine a real company — Olist, a Brazilian online marketplace — hands you a pile of messy data: about 100,000 customer orders spread across nine separate files, covering 2016 to 2018. Some delivery dates are missing. Some orders have duplicate reviews. Product categories are written in Portuguese. Payment totals don't always match order totals to the cent. Before annual planning, three company leaders each need a straight answer from this data:

- **The CFO asks:** where does our money come from, and is it growing?
- **The marketing head asks:** do customers ever come back and buy again?
- **The Head of Operations asks:** does slow delivery actually hurt customer satisfaction?

This project answers all three the way a professional analytics team would: it cleans and organizes the messy data into a proper database, runs a real statistical test, produces an honest 28-day sales forecast, builds an interactive dashboard, and closes with a one-page memo of findings and recommendations.

1.1 Why This Project Is Well-Built

- Real, messy data across 9 related files — not one tidy spreadsheet.
- A proper data warehouse: a verbatim 'staging' copy of the raw data, then a cleaned 'star schema' — the industry-standard structure.
- The data's honesty is checked at every step, and every problem found is recorded, not hidden.
- A real statistical test (not just a chart) proves the delivery-satisfaction link.
- The forecast is checked against a simple baseline, and the project honestly reports when that baseline wins.
- Anyone can rebuild the entire project from scratch with a handful of commands — it is fully reproducible.

1.2 The Dataset and Its Three Traps

The Brazilian E-Commerce Public Dataset by Olist (Kaggle slug: olistbr/brazilian-ecommerce) has nine CSV files: orders, order items, payments, reviews, customers, sellers, products, a category-name translation table, and geolocation. Three things make it a genuinely good analytics dataset rather than a toy example:

1. **customer_id vs. customer_unique_id:** customer_id is unique per ORDER, not per person — a returning customer gets a brand-new customer_id every time they order. Only customer_unique_id identifies the actual person. Get this wrong and the repeat-purchase rate falsely reads as roughly 0%.
2. **Items total ≠ payments total:** item-line revenue and payment records don't always match exactly (vouchers, installments, cancellations). The project quantifies this gap instead of ignoring it.
3. **Multiple reviews and impossible timestamps:** some orders have duplicate review rows, and some have delivery dates before ship dates. The project de-duplicates reviews (latest wins) and logs the temporal anomalies rather than silently dropping them.

1.3 Headline Findings

These are the project's own computed, saved results — reproducible by running the cited SQL or notebook. None are estimated.

Finding	Result
Repeat-purchase rate	3.0% — only 2,801 of 93,358 customers ever order twice
Late delivery vs. review score	Late: median 2/5 (54.07% rated 1-2 stars). On-time: median 5/5 (9.22%). Tested with a one-sided Mann-Whitney U test ($p < 0.001$)
Share of orders delivered late	8.11% — 7,826 of 96,478 delivered orders
Revenue concentration	Top 5 of 74 categories generate 39.25% of revenue
Payment reconciliation	99.70% of orders reconcile exactly; the 0.30% gap is fully explained
RFM 'Champions'	1.29% of customers generate 2.52% of revenue
28-day demand forecast	6,660 orders expected; the daily 95% bounds add up to 3,585-9,943, which is not a 28-day interval; the simple baseline was as accurate as the more complex model

⚠️ **NOTE** Full sourcing and exact figures for every finding are in Section 25 (Verifying Your Results) and in the project's own docs/findings.md.

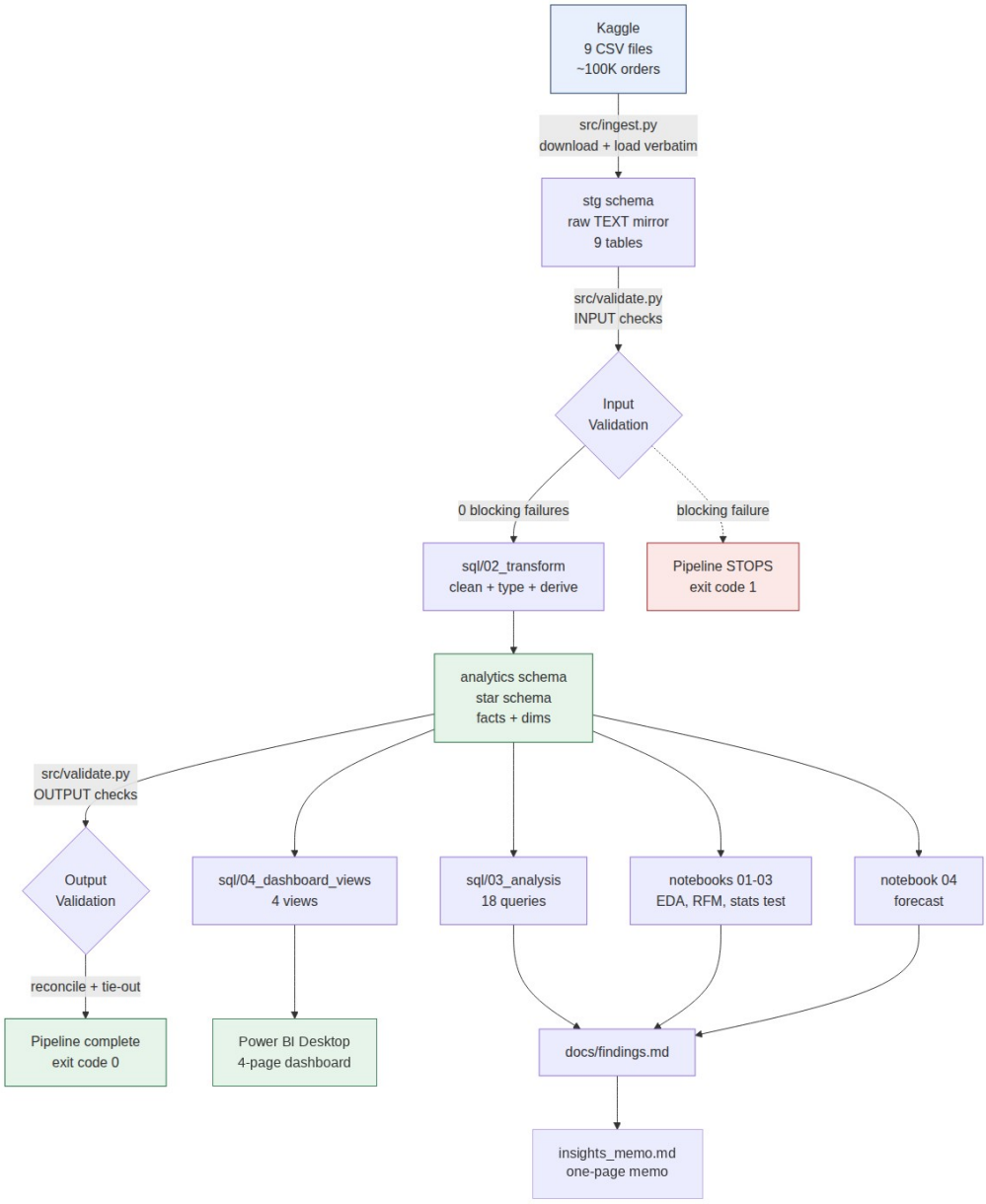
1.4 What the Findings Mean for the Business

Each finding answers one of the three leadership questions. The right-hand column matters as much as the middle one: it says what the finding cannot tell you, so that nobody over-reads it. All findings come from one marketplace between 2016 and 2018 and are observational: they describe what happened, not what any change would cause.

Finding	What it suggests	What it does not show
Repeat-purchase rate 3.0% (2,801 of 93,358 customers)	Growth depends on winning new customers. Retention work would start from a very small base (month-1 retention is 0.48%).	Why customers do not return, or whether they bought from other sellers elsewhere.
Top 5 of 74 categories = 39.25% of revenue	A few categories carry much of the revenue, so stock and delivery in those matter most. The other 69 share the remaining 60.75%.	Profit. Revenue here is item price plus freight, with no costs.
Late deliveries 8.11%; 54.07% of late orders rated 1-2 stars, against 9.22% of on-time orders	Delivery reliability is strongly linked to unhappy customers, so it is a sensible place for Operations to look first.	That lateness causes the low ratings. Distance, seller and freight cost are plausible confounders.
Payment gap 0.30% (303 of 99,441 orders)	Item-level revenue is reliable: it ties out to staging exactly. The small gap with payments comes from vouchers, instalments and cancellations.	That payments received equal revenue. Payments include every order status.
RFM: Lapsed One-Time is 58.29% of customers and 55.88% of revenue; Champions are 1.29% and 2.52%	Most revenue comes from people who bought once and have not returned. The already-loyal group is tiny.	What any segment will do next. Segments describe past behaviour.
28-day forecast of 6,660 orders	A simple repeat-last-week model is as accurate as Holt-Winters (WAPE 0.2308 against 0.2310), so use the simple one for planning.	A prediction interval for the 28-day total. In the six backtest folds the 28-day totals missed by between -2,363 and +731 orders.

2. Architecture & How Data Flows

The whole project is one directional pipeline: raw files go in one end, a governed database and set of analyses come out the other. Every stage is a script that can be re-run safely.



1. **Ingest** — download the dataset and load all 9 files verbatim into stg (staging).
2. **Validate input** — check the raw data. Blocking problems stop the run; minor, expected issues are logged and allowed.
3. **Transform** — clean, type, and reshape stg into the analytics star schema.
4. **Validate output** — confirm the built database reconciles (row counts, no broken links, revenue ties out).

2.1 The Two Database Areas

Schema	Contents	Purpose
stg (staging)	An exact, untouched copy of the 9 source CSVs — every column stored as plain text	Traceability: any data problem can be traced back to the original file
analytics	Cleaned, typed tables organized as a star schema (facts + dimensions)	The version every query, notebook, and dashboard actually uses

2.2 Two Populations That Are Never Mixed

✖ **WARNING** Revenue, retention, and repeat-purchase analysis use DELIVERED orders only (order_status = 'delivered'). The demand forecast uses ALL placed orders, because operations must plan capacity for every order, not only the ones that succeeded. Mixing these two populations in one chart would silently distort both answers — the project keeps them strictly separate throughout.

2.3 Two Environments

Environment	What runs there	What data it uses
Your computer (local)	The full project — pipeline, notebooks, dashboard	Real Olist data (~100,000 orders) from Kaggle
GitHub Actions (CI)	Automated tests, on every push	Small, fake 'fixture' data (~100 rows), so no Kaggle login is needed

⚠ **NOTE** This is why the automated tests use tiny fake data while your computer uses the real dataset — both are correct, for different purposes. The notebooks only run correctly on the real data, which is why CI does not run them.

3. Technology Stack

Every tool below is actually used in this project — nothing here is aspirational.

Technology	What it is	Role here
Python 3.12	A general-purpose programming language	Runs the entire pipeline: download, load, validate, transform
Virtual environment (.venv)	An isolated, project-specific copy of Python and its libraries	Keeps this project's dependencies separate from anything else on your machine
Git & GitHub	Version control and code hosting	The project's source of truth; you clone it to your machine
VS Code	A code editor	Where you browse code, run notebooks, and use the terminal
Docker Desktop	Runs software in isolated containers	Runs PostgreSQL without installing it directly
Docker Compose	Describes how to run one or more containers	Defines exactly how the PostgreSQL container starts
PostgreSQL 16	A free, professional relational database	Stores every stage of the data and runs all SQL
SQL	The language for querying and shaping data	All data cleaning and all 18 analysis queries

Technology	What it is	Role here
Make (GNU Make)	Runs project commands using short names	Every workflow step is one 'make' command
Jupyter Notebook	An interactive document mixing code, results, and text	The 4 analysis notebooks
Power BI Desktop	A free Microsoft dashboard-building tool	Builds the 4-page interactive dashboard (Windows only)
DAX	Power BI's formula language	Defines the dashboard's KPI calculations
CSV files	Plain, comma-separated data files	The Kaggle dataset's format, and every saved query output
.env file	A small text file holding local settings and secrets	Database host, port, name, username, password — never committed

3.1 Python Libraries Used

Library	Version	What it does
pandas	2.2.3	Reads, reshapes, and analyzes tables of data
SQLAlchemy	2.0.51	Connects Python to PostgreSQL
psycopg2-binary	2.9.12	The driver SQLAlchemy uses to talk to PostgreSQL
scipy	1.13.1	Provides the Mann-Whitney statistical test
statsmodels	0.14.6	Provides the Holt-Winters forecasting model
matplotlib	3.9.4	Draws the notebook charts
python-dotenv	1.2.2	Reads settings from .env
pytest	8.4.2	Runs the 10 automated tests
jupyter / nbconvert / nbformat	1.1.1 / 7.17.1 / 5.10.4	Runs and manages the notebooks
kaggle	2.2.3	Downloads the dataset
openpyxl	3.1.5	Writes the Excel summary file

4. What Exists — and What Does Not

An honest inventory. Some components a reader might expect are deliberately not part of this project.

ⓘ **NOT IN THIS PROJECT** *There is no Dockerfile. The project uses Docker Compose with the official, unmodified PostgreSQL 16 image — simpler and correct for running a standard database, with no custom image to build or maintain.*

ⓘ **NOT IN THIS PROJECT** *There is no Power Query (M) transformation layer. All reshaping happens earlier, in SQL views (sql/04_dashboard_views/). Power BI imports those finished views and tables with a plain native SELECT (Section 19.4).*

⚠️ **NOTE** The dashboard exists: dashboard/ecommerce_retention_dashboard_final.pbix. Part V describes what it contains and how to refresh or rebuild it.

PART II — INSTALLATION & SETUP

This part takes you from an empty computer to a fully running project. Follow every section in order. After every action, this guide tells you exactly what you should see, so you can confirm the step worked before moving on.

✔️ **SUCCESS** Time budget: ~10-15 min for setup (Sections 5-9) + ~5-10 min for the pipeline (Section 11) + ~5-15 min for tests and notebooks (Sections 12, 18) + a few minutes to open the finished dashboard in Power BI Desktop (Part V, optional). Disk/network: ~1GB free disk (Postgres image + data + .venv), ~50MB Kaggle download, ~500MB more if you also install Power BI Desktop.

Box	Meaning
STEP	An action to perform
✔️ SUCCESS	What you should see when a step works correctly
⚠️ NOTE	Important information worth knowing, not urgent
✖️ WARNING	A hard warning — getting this wrong breaks something, not just an inconvenience
ⓘ NOT IN THIS PROJECT	Flags something that is not part of this project
HOW TO VERIFY	A specific check confirming a whole section succeeded

5. Prerequisites

5.1 Hardware

Component	Minimum	Why
RAM	8 GB (16 GB recommended)	Docker, Python, and Power BI run at the same time
Disk space	5 GB free	Docker image, dataset, Python libraries, Power BI
Internet	Required for setup	Downloads tools, libraries, and the dataset

5.2 Software You Must Install Yourself

Software	What it is	Where to get it
Docker Desktop	Runs the database	docker.com/products/docker-desktop
Python 3.12	Runs the pipeline	python.org/downloads
Git	Downloads the project code	git-scm.com/download/win
Power BI Desktop	Opens and refreshes the dashboard (Part V)	Microsoft Store — search 'Power BI Desktop'

⚠️ **NOTE** This guide covers configuring Docker for the project, not installing Docker Desktop itself. If it's missing, install it from the link above first.

5.3 Verify Your Prerequisites

Open PowerShell and run each check:

```
docker --version
python --version
git --version
```

✔️ **SUCCESS** Each should print a version number (Python 3.11 or 3.12 is fine). If any says 'not recognized', install it before continuing.

6. Get the Project Code

⚠️ **NOTE** Git: preinstalled on macOS and most Linux distros; Windows users get it from git-scm.com/download/win.

This example uses D:\PROJECTS — use any folder you like, e.g. C:\PROJECTS if you don't have a D: drive.

```
mkdir D:\PROJECTS
cd D:\PROJECTS
git clone https://github.com/MARCUS-00/ecommerce-retention-analytics.git
cd ecommerce-retention-analytics
```

HOW TO VERIFY dir should show:

Makefile, README.md, docker-compose.yml, requirements.txt, and folders named src, sql, notebooks, docs, tests, dashboard and data. Also present: LICENSE, .gitignore, .env.example and a .github folder.

7. Install make

What is make? A small tool that runs the project's commands with short names ('make pipeline' instead of a long Python command). Windows doesn't include it.

⚠️ **NOTE** macOS/Linux: make is preinstalled or one line away — apt install make (Debian/Ubuntu) or brew install make (macOS). The rest of this section is Windows-specific.

7.1 Recommended: Scoop (no admin rights needed)

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
Invoke-RestMethod -Uri https://get.scoop.sh | Invoke-Expression
scoop install make
```

7.2 Alternative: Chocolatey (needs an elevated shell)

Open PowerShell as Administrator, then:

```
choco install make -y
```

✖️ **WARNING** Chocolatey requires an elevated window. If you get a 'not running from an elevated shell' message, reopen PowerShell as Administrator. A lock-file error means a prior attempt crashed — delete the named lock file and retry, or use Scoop instead.

HOW TO VERIFY make --version should show GNU Make 4.x.

If not found, open a brand-new terminal window — PATH updates only apply to new windows.

8. Set Up Kaggle Credentials

The dataset lives on Kaggle. The project downloads it automatically once your free Kaggle credentials are in place.

1. Sign up free at kaggle.com.
2. Profile picture (top-right) → Settings → API section → Create New Token.
3. A file named kaggle.json downloads — remember where (usually Downloads).

```
mkdir "$env:USERPROFILE\.kaggle" -ErrorAction SilentlyContinue
Copy-Item "$env:USERPROFILE\Downloads\kaggle.json" "$env:USERPROFILE\.kaggle\
kaggle.json"
```

✖ **WARNING** Never share kaggle.json and never put it inside the project folder — it contains a secret key. If a token is ever exposed (a screenshot, a shared screen), revoke and regenerate it immediately at kaggle.com → Settings → API → Expire Token.

HOW TO VERIFY kaggle datasets list --max-size 1 should return a small table of dataset names.

If 'kaggle' is not recognized, the Kaggle CLI installed to a Scripts folder not on PATH — a fix already proven to work on this project. Run python --version first to check your version, then substitute the matching folder name (e.g. Python311 for 3.11, Python312 for 3.12):

```
$env:Path = "C:\Users\<you>\AppData\Roaming\Python\Python<your version>\
Scripts;$env:Path"
```

9. Configure the Project

9.1 Environment Variables (.env)

```
@"
DB_HOST=localhost
DB_PORT=5432
DB_NAME=olist
DB_USER=analyst
DB_PASSWORD=analyst
"@ | Out-File -FilePath .env -Encoding ascii
```

Setting	Meaning
DB_HOST	The database runs on your own computer (localhost)
DB_PORT	PostgreSQL's standard port
DB_NAME	The database name (olist)
DB_USER / DB_PASSWORD	Local development credentials

✖ **WARNING** Never commit .env — it's already gitignored, so a normal 'git add .' won't pick it up. Don't override that.

✖ **WARNING** Set .env BEFORE the first 'make db-up', not after. These values seed the PostgreSQL container on its very first boot (via POSTGRES_USER/POSTGRES_PASSWORD/POSTGRES_DB in docker-compose.yml). Editing .env after the container's data volume already exists does NOT change the running database's actual password — you'll need docker compose down -v (destroys and rebuilds the database) for a changed password to take effect. This exact mismatch is a real, documented cause of Power BI authentication failures (Section 22).

9.2 docker-compose.yml, Explained

This file tells Docker exactly how to run the database, using the official PostgreSQL 16 image — no custom image is built.

Setting	Value	Why
image	postgres:16	The official, unmodified PostgreSQL 16 image
POSTGRES_USER/ PASSWORD/DB	from .env	Sets credentials from your local configuration
POSTGRES_INITDB_ARGS	--auth-host=md5	PostgreSQL 16 defaults to scram-sha-256, which its documentation calls the most secure password method but notes is not supported by older client libraries. Power BI Desktop rejected valid credentials on this project until md5 was used (Section 19.3). PostgreSQL's documentation says md5 is no longer considered secure, so this is acceptable for a local development database only, never for a shared or production one.
ports	${DB_PORT}:5432	Exposes the database on your machine's port 5432
volumes	olist_pg_data	Persists data across restarts
healthcheck	pg_isready	Docker repeatedly checks the database is actually ready

10. Start the Database & Install Dependencies

✖ **WARNING** Open Docker Desktop and wait for 'Engine running' (steady green icon) before continuing.

STEP 1 Install Python libraries — ~1-3 minutes

```
make setup
```

Runs: python -m venv .venv, then pip install --upgrade pip, then pip install -r requirements.txt.

✔ **SUCCESS** No 'ERROR' line by the end = success. A new .venv folder now exists.

STEP 2 Start the database container

```
make db-up
```

Runs: docker compose up -d --wait.

HOW TO VERIFY docker compose ps should show olist_postgres as 'healthy'.

'starting' means wait a few seconds and check again.

⚠ **NOTE** Activating the virtual environment (.venv\Scripts\Activate.ps1) is only needed if you want to run python, jupyter, or a one-off script by hand. Every 'make' target already calls .venv's Python directly by full path (see the Makefile's VENV_PY variable) and does not need the shell to have the environment activated first.

11. Run the Pipeline

This is the heart of the project. It downloads the real data (~50 MB), loads it, checks it, and builds the analytics warehouse. Requires internet access; takes 3-8 minutes the first time.

STEP 1 Run the pipeline

```
make pipeline
```

Expected output pattern — this exact pipeline, run end to end against the full real dataset, produces this log shape. Row counts are noted <N> because they scale with whichever CSV snapshot Kaggle serves; the anomaly and pipeline-status lines are fixed and verified:

```
INFO src.ingest: Loaded stg.orders: <N> rows (from olist_orders_dataset.csv)
   ...one line per source file, 9 total...
WARNING src.validate: LOGGED [temporal_sanity] orders: ... (count=<N>)
INFO src.validate: Input validation: PASSED (0 blocking failures, 9 logged anomalies)
INFO src.transform: Executed 01_create_schema.sql (1 statements)
   ...one line per SQL file in sql/01_schema and sql/02_transform...
WARNING src.validate: LOGGED [revenue_tie_out] ... (count=<N>)
INFO src.validate: Output validation: PASSED (0 blocking failures, 1 logged anomalies)
INFO __main__: Pipeline complete
```

✔️ **SUCCESS** Success: the final line reads exactly 'Pipeline complete'. WARNING/LOGGED lines are expected, documented anomalies — not errors. The pipeline only stops (exit code 1) on a blocking check.

✖️ **WARNING** A Kaggle authentication error mid-run means an expired token. Regenerate at kaggle.com → Settings → API → Create New Token, replace ~/.kaggle/kaggle.json, and re-run.

STEP 2 Build the dashboard views and Excel summary

```
make views
make excel
```

✔️ **SUCCESS** 'views' ends with 'Dashboard views built'; 'excel' writes dashboard/exec_summary.xlsx.

✔️ **SUCCESS** After 'make views' the four views hold these row counts on the dataset snapshot used for this project: vw_sales 110,197; vw_orders 96,478; vw_customers 93,358; vw_daily_orders 634. Check one with SELECT COUNT(*) FROM analytics.vw_orders; (Section 14.7 shows how to run a query).

⚠️ **NOTE** The 'Raw Extract' sheet in exec_summary.xlsx holds the first 5,000 rows of an unordered query (src/build_excel_summary.py), so its rows can differ from run to run. It is a sample, not an exact extract. The pivot sheets are exact and identical between runs.

12. Run the Tests

The project includes 10 automated tests confirming the database was built correctly.

```
make test
```

Verified expected output — timing varies by machine; measured 3-6s on a run against the full dataset:

```
..........                                                          [100%]
10 passed in 3-6s
```

✔️ **SUCCESS** '10 passed' = your installation is verified and correct.

PART III — UNDERSTANDING THE CODE

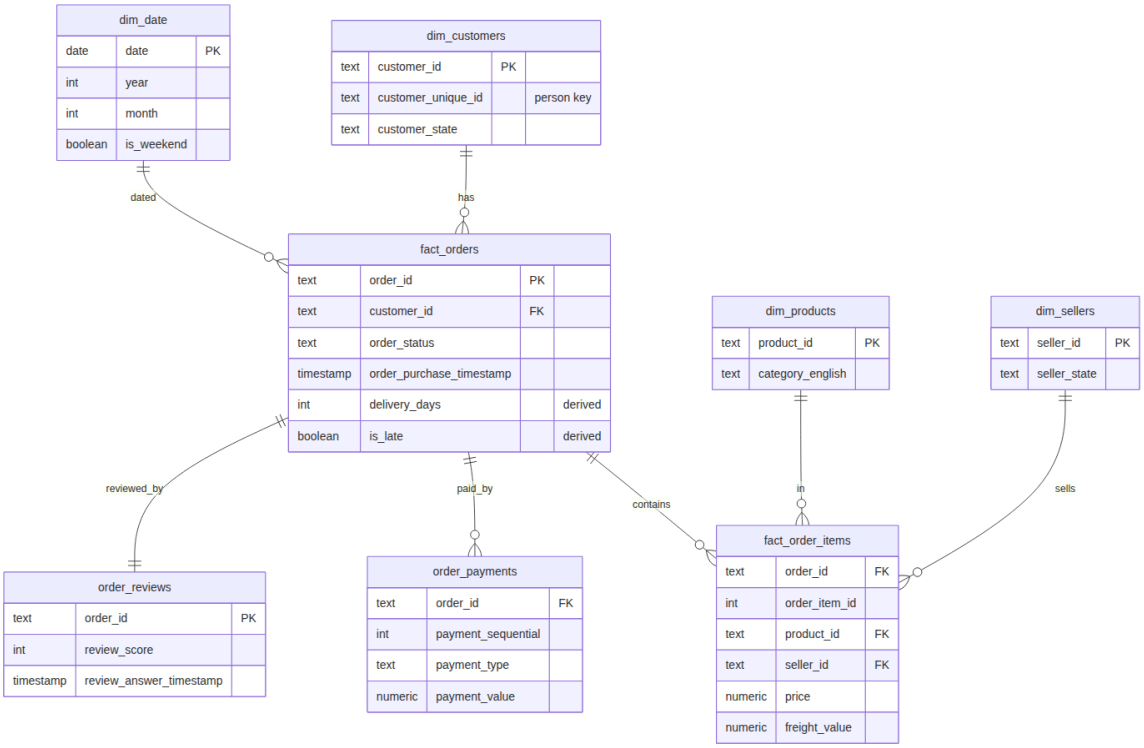
13. Project Structure

The folder layout after cloning (caches and the virtual environment omitted):

```
ecommerce-retention-analytics/
├── README.md                Project overview and headline findings
├── Makefile                 Every project command
├── requirements.txt         Pinned Python library versions
├── docker-compose.yml       Defines the PostgreSQL container
├── .env.example             Template for local configuration
├── .env                     Your actual local configuration (never committed)
├── .gitignore               Files Git must never track
├── LICENSE                  MIT license (code only)
│
├── docs/
│   ├── MASTER_DOC.md, guides/         Design record (planning); PDF guides
│   ├── PLAYBOOK.md                    Setup/run walkthrough (troubleshooting,
│   │                                  DB inspection, Power BI build)
│   ├── findings.md                    7 verified findings, with sources
│   ├── insights_memo.md               One-page executive memo (Section 20)
│   ├── data_dictionary.md             Every table and column, plus the anomaly log
│   ├── architecture.md                How data flows through the system
│   └── monitoring_policy.md           When and how to retrain the forecast model
│
├── sql/
│   ├── 01_schema/           Creates the analytics schema + calendar table (2 files)
│   ├── 02_transform/        Builds the star schema from staging (8 files)
│   ├── 03_analysis/         18 business-question queries + saved outputs
│   └── 04_dashboard_views/ 4 views that feed Power BI
│
├── src/                     8 Python modules (ingest, validate, transform, ...)
├── notebooks/               4 Jupyter notebooks
├── dashboard/
│   ├── ecommerce_retention_dashboard_final.pbix    Final Power BI report
│   ├── archive/                                    Superseded files (local only)
│   ├── screenshots/                                One screenshot per dashboard page
│   ├── BUILD_QA_REPORT.md                          Checks, decisions and limitations
│   ├── DAX_MEASURES_FINAL.md                       43 measures + calculated column
│   ├── BUILD_SPEC.md, dax_measures.md              Original specification
│   └── exec_summary.xlsx                           Excel summary (built by make excel)
│
├── tests/                           10 automated tests + synthetic fixtures for CI
├── data/raw/                        Real Olist CSVs (gitignored)
├── data/processed/forecast_28d.csv  28-day forecast written by notebook 04
└── .github/workflows/ci.yml         Automated tests on every GitHub push
```

14. Database & SQL Reference

The analytics schema is a 'star schema': fact tables (things that happened) surrounded by dimension tables (descriptions) — the standard professional layout for analytical databases.



14.1 Fact Tables

Table	Grain	Key columns
fact_orders	One row per order	order_id (PK), customer_id (FK), order_status, purchase/delivery timestamps, delivery_days and is_late (both derived, NULL for undelivered orders)
fact_order_items	One row per item line	order_id + order_item_id (PK), product_id (FK), seller_id (FK), price, freight_value
order_payments	One row per payment record (an order can have several)	order_id + payment_sequential (PK), payment_type, payment_value
order_reviews	One row per order (de-duplicated)	order_id (PK), review_score, review_answer_timestamp

14.2 Dimension Tables

Table	Grain	Notable columns
dim_customers	One row per customer_id (order-scoped)	customer_unique_id — the PERSON key
dim_products	One row per product	product_category_name_english (translated from Portuguese)
dim_sellers	One row per seller	seller_state
dim_date	One row per calendar day, 2016-2018	date, year, month, weekday, is_weekend
geolocation	One row per zip prefix (de-duplicated)	avg_lat, avg_lng, city, state

✖ **WARNING** The single most important rule in this database: customer_id is unique per ORDER, not per person — a returning customer gets a new customer_id on every order. The real person is identified by customer_unique_id. Every retention, repeat-purchase, and RFM calculation in this project uses customer_unique_id. Using customer_id instead makes the repeat-purchase rate falsely read as roughly 0%.

14.3 SQL Files by Folder

Folder	Files	Purpose
sql/01_schema	2	Creates the analytics schema and the calendar (dim_date) table
sql/02_transform	8	Cleans and reshapes staging data into the star schema
sql/03_analysis	18	Answers the business questions; each saves a sample output CSV
sql/04_dashboard_views	4	Builds the views Power BI imports

14.4 The 18 Analysis Queries, by Stakeholder

File	Answers
01_monthly_revenue_mom_growth.sql	Monthly revenue and month-over-month growth
02_aov_trend.sql	How average order value changes month to month
03_category_pareto.sql	Revenue share per category (top 5 of 74 = 39.25%)
04_revenue_by_state.sql	Revenue concentration by Brazilian state
05_payment_mix_installments.sql	Payment method breakdown and installment reliance
06_seller_concentration.sql	Revenue concentration among the top 10 sellers
07_order_status_funnel.sql	How orders distribute across statuses (all placed orders)
08_delivery_days_distribution.sql	The spread of delivery times
09_late_pct_by_month.sql	Late-delivery rate trend by month (overall: 8.11%)
10_late_pct_by_state.sql	Which states have the highest late rates
11_late_vs_review_score.sql	Review scores for late vs. on-time orders
12_cohort_retention_matrix.sql	Monthly cohort retention (person-level, delivered orders)
13_repeat_purchase_rate.sql	Share of customers who buy twice (3.0%)
14_rfm_customer_scores.sql	Recency/Frequency/Monetary score per customer
15_rfm_segment_share.sql	Customers and revenue per RFM segment
16_items_vs_payments_reconciliation.sql	Orders where item and payment totals differ (0.30%)
17_duplicate_review_count.sql	Orders with more than one review before de-duplication
18_temporal_violation_count.sql	Orders with impossible timestamp orderings

14.5 Example Query — Repeat-Purchase Rate

Quoted exactly from sql/03_analysis/13_repeat_purchase_rate.sql — the query behind the 3.0% headline finding:

```
-- QUESTION: What share of customers ever place more than one delivered order?
WITH orders_per_customer AS (
    SELECT
        c.customer_unique_id,
        COUNT(DISTINCT o.order_id) AS delivered_orders
    FROM analytics.fact_orders o
    JOIN analytics.dim_customers c USING (customer_id)
    WHERE o.order_status = 'delivered'
    GROUP BY 1
)
SELECT
    COUNT(*) AS total_customers,
    COUNT(*) FILTER (WHERE delivered_orders >= 2) AS repeat_customers,
    ROUND(100.0 * COUNT(*) FILTER (WHERE delivered_orders >= 2) / COUNT(*), 2) AS
repeat_pct
FROM orders_per_customer;
```

14.6 Verifying the Database Directly

```
SELECT 'fact_orders' AS table, COUNT(*) FROM analytics.fact_orders
UNION ALL SELECT 'fact_order_items', COUNT(*) FROM analytics.fact_order_items;
```

On real data, fact_orders returns 99,441 rows, matching the source orders file exactly.

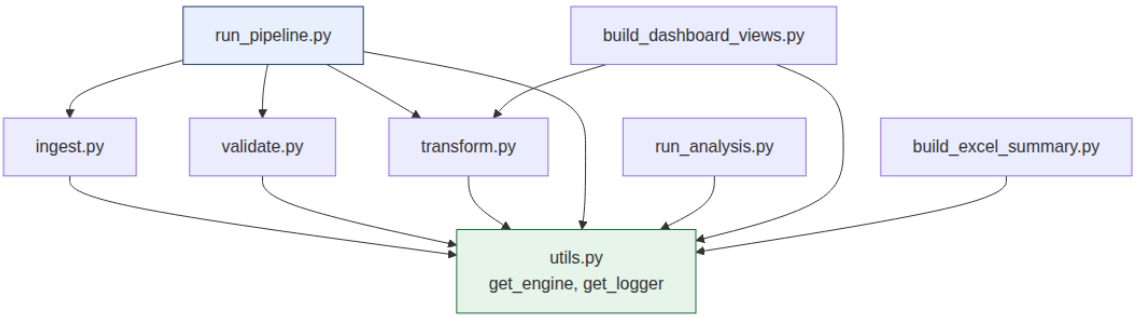
14.7 Inspecting the Database Directly (Troubleshooting Tool)

Useful whenever something upstream (Power BI, a notebook, a query) looks wrong and you need to know if the problem is the database itself or the tool on top of it. Connect with psql — bundled inside the PostgreSQL Docker image, nothing extra to install:

```
docker exec olist_postgres psql -U analyst -d olist -c "SELECT COUNT(*) FROM
analytics.fact_orders;"
```

✔️ **SUCCESS** Expect 99441. If this fails or returns something else, the problem is in the database itself, not in whatever tool you were using on top of it — that narrows down where to keep looking. Any GUI client (DBeaver, pgAdmin, or Power BI's own preview pane) works the same way for browsing tables interactively.

15. Python Code Reference



Module (src/)	Role
utils.py	Shared database connection and logger — every other module depends on this
ingest.py	Downloads the dataset from Kaggle and loads it into staging
validate.py	All data-quality checks — both on raw staging and on the built analytics schema

Module (src/)	Role
transform.py	Executes the SQL that builds the star schema (and the dashboard views)
run_pipeline.py	The single entry point — orchestrates ingest → validate → transform → validate
run_analysis.py	Runs all 18 analysis queries and saves a sample of each result
build_dashboard_views.py	Builds the 4 views used by Power BI (vw_sales, vw_orders, vw_customers, vw_daily_orders). analytics.forecast_28d is a separate table, written by notebooks/04_demand_forecast.ipynb — not one of these views.
build_excel_summary.py	Builds the 3-sheet Excel summary workbook

15.1 Validation Checks in Detail

Check (input, on staging)	Severity	Confirms
Row counts	Blocking	Every staging table has at least one row
Primary key uniqueness	Blocking	No duplicate IDs where there should be none
Referential integrity	Blocking	Every linked ID exists in its parent table
Domain values	Blocking	Review scores are 1-5; prices are not negative
Temporal sanity	Logged	Impossible date orderings are counted (they exist in source data)
Null policy / review duplicates	Logged	Expected null rates and duplicate reviews are counted

Check (output, on analytics)	Confirms
Row reconciliation	Analytics table counts match staging after documented de-duplication
Orphan foreign keys	Every link in the analytics schema points to a real row
Revenue tie-out	Total item revenue in analytics exactly equals the staging total

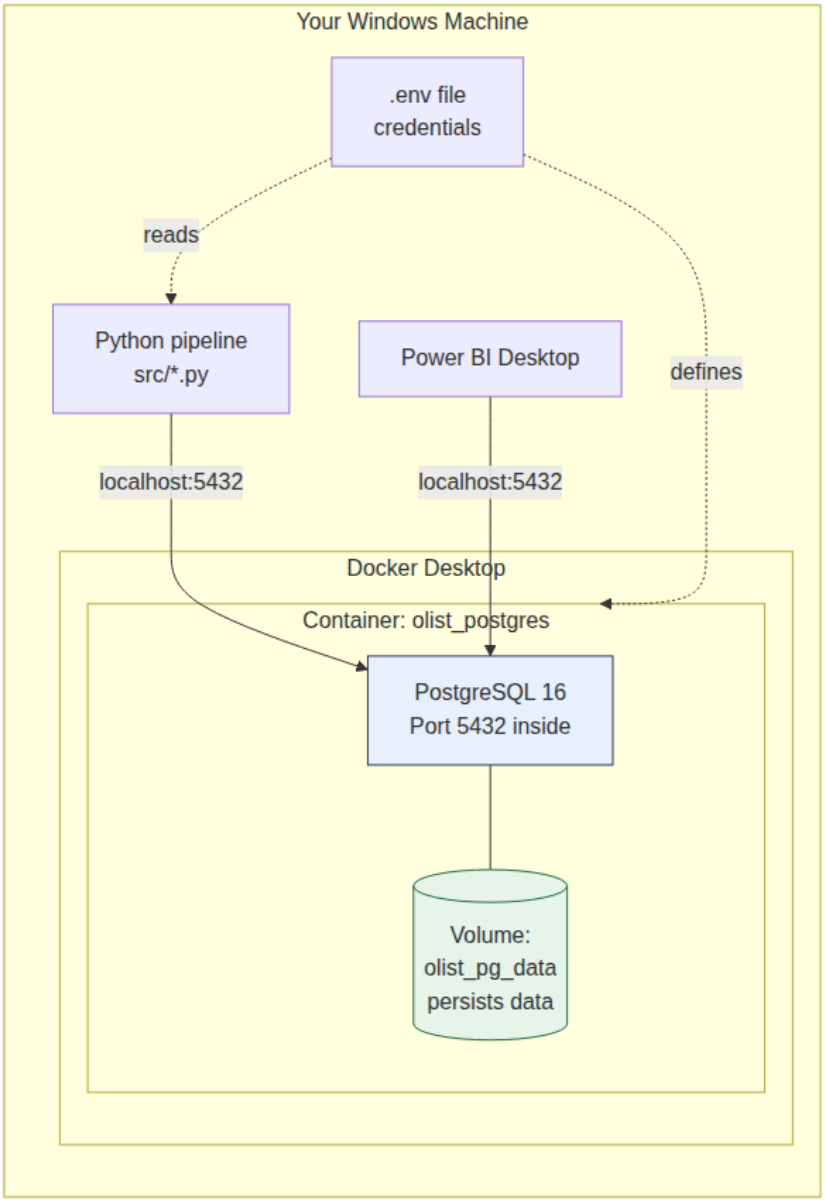
15.2 Verified Code Quality

All 11 Python files (8 in src/, 3 in tests/) were checked once with these four tools in October 2026, outside the project's normal workflow:

Tool	Checks for	Result
py_compile	Syntax errors	All 11 files compile
pyflakes	Undefined names, unused imports	Clean
ruff	Comprehensive linting	Clean with the default error rules (E4, E7, E9, F); newer rule sets add style findings
vulture	Dead (unused) code	None found

⚠️ **NOTE** None of pyflakes, ruff, or vulture are in this project's requirements.txt, CI pipeline, or config — there's no ongoing lint step. Add a ruff dev-dependency and a CI job if you want this enforced going forward; it doesn't exist today.

16. Docker Reference



A container is a self-contained, isolated mini-environment — like a sealed lunchbox with everything a program needs inside it, not interfering with the rest of your machine, disposable and recreatable at will.

Concept	Plain-English meaning
Image	A ready-made template for a container (here: PostgreSQL 16)
Container	A running copy of an image (here: olist_postgres)
Volume	Storage that survives even when the container is deleted
Port	A numbered 'door' programs use to talk to the container (here: 5432)
Docker Compose	The tool that reads docker-compose.yml and starts the container correctly

Command	What it does
make db-up	Starts the container (docker compose up -d --wait)
make db-down	Stops the container (docker compose down)
docker compose ps	Shows whether the container is running and healthy
docker compose logs postgres	Shows the database's log messages
make clean	Stops the container AND deletes its volume — erases all data

17. Developer Guide

For anyone modifying or extending the code. Preserve these rules from the project's design specification:

Rule	Why it matters
No fabricated numbers	Every figure in any document must be computed by the code, or removed
Staging is verbatim	stg is an exact TEXT copy — no cleaning. All cleaning happens in sql/02_transform, keeping every anomaly traceable
Person key vs. order key	All retention/RFM math uses customer_unique_id, never customer_id
Two populations, never mixed	Revenue/retention use delivered orders only; the forecast uses all placed orders
Idempotent everything	Every script uses DROP IF EXISTS + CREATE, so re-running is always safe
Time-series validation is walk-forward	The forecast never shuffles time into random folds — that leaks future data into the past

17.1 How to Add a New Analysis Query

1. Create sql/03_analysis/19_your_question.sql, starting with a '-- QUESTION:' comment.
2. Add WHERE order_status = 'delivered' for revenue/retention; omit it for all-placed-order analysis.
3. Join through customer_unique_id for any customer-level count.
4. Run python -m src.run_analysis — it runs every query in the folder and saves a sample output.

17.2 Testing & CI

tests/test_input_quality.py checks staging data; tests/test_output_quality.py checks analytics data. Run both with make test. tests/fixtures/generate_fixtures.py creates small synthetic data for CI, deliberately planting three defects (a duplicate review, a payment mismatch, a temporal violation) so validation is genuinely exercised, not trivially green.

.github/workflows/ci.yml runs on every GitHub push: starts a PostgreSQL 16 service container, installs pinned libraries, generates fixtures, runs the pipeline, then pytest. CI does not run the notebooks — they need the full real dataset, which local development supplies but CI does not.

PART IV — RUNNING THE ANALYSIS

18. Notebook Guide

Run all four with 'make notebooks' (this runs all four, including the forecast, in one command), or open any one individually in VS Code and select Run All. They read from the analytics database, so the pipeline must be run first.

Notebook	Purpose	Key output
01_eda.ipynb	Explore and profile the raw data	Order-volume chart; the stable date window used by the forecast
02_retention_rfm.ipynb	Customer retention and segments	Cohort heatmap; RFM segment chart
03_delivery_reviews.ipynb	Statistical test on delivery vs. reviews	Mann-Whitney test result
04_demand_forecast.ipynb	28-day order forecast	Forecast chart; writes analytics.forecast_28d

18.1 Notebook 01 — Exploratory Data Analysis

Profiles order volume over time, null timestamps by status, category/state distributions, and delivery-days spread. It also derives the 'stable window' (2017-01-05 to 2018-08-22, the range with no gap days) that notebook 04 reuses for the forecast.

> ⚠️ **NOTE** This notebook has a built-in check that the stable window has no gap days. Against tiny test fixtures (100 rows) this correctly fails, because 100 rows cannot cover every calendar day — that is the check working, not a bug. On real data it passes.

18.2 Notebook 02 — Retention Cohorts & RFM

Produces the cohort retention heatmap (near-empty — confirming the 3.0% repeat rate visually) and RFM segmentation, scoring each customer on Recency, Frequency, and Monetary value into segments: Champions, At-Risk Repeat, Recent One-Time, Lapsed One-Time.

18.3 Notebook 03 — Delivery vs. Reviews (Statistical Test)

Uses a Mann-Whitney U test — the correct test because review scores are ordinal (1-5), not continuous — comparing late vs. on-time order score distributions:

Group	Median score	% rated 1-2 stars	Count
Late orders	2 / 5	54.07%	7,661
On-time orders	5 / 5	9.22%	88,163

$p < 0.001$. The notebook prints 0.000e+00 because the exact value is too small for a floating-point number; that is not a probability of zero. A gap this large is very unlikely to be chance, and at this sample size the effect size (median 2 vs. 5, 54.07% vs. 9.22% rated 1-2 stars) matters more than the p-value. The test was re-run from the raw CSV files in October 2026 and reproduced n and U exactly. The notebook also notes this is an association, not proof of cause: late delivery correlates with distance, seller, and freight cost.

18.4 Notebook 04 — 28-Day Demand Forecast

Compares two forecasting approaches using walk-forward validation — training only on past data to predict the future, so the model never sees the future during testing:

Model	Mean WAPE (lower is better)	Verdict
Seasonal naive (repeat last week)	0.2308	Winner — simpler and just as accurate
Holt-Winters (statistical model)	0.2310	Effectively tied; did not beat the baseline

The honest result: the simple baseline was as accurate as the complex model, so the project recommends the simple one. Forecast: 6,660 orders over the next 28 days. Each day has 95% empirical bounds, and over 28 days those bounds add up to 3,585-9,943; that sum is not a 28-day prediction interval, because adding daily bounds assumes the daily errors move together perfectly. This notebook also keeps the Black Friday 2017 spike entirely inside the training data of every test fold, so it cannot distort reported accuracy.

✖ **WARNING** Notebook 04 writes analytics.forecast_28d as a side effect. This table must exist before the Power BI forecast band (Section 19.7, Page 4) will show anything — if you build the dashboard before running this notebook, that visual will be empty.

18.5 Confirming Real Data Was Used

Notebook	Look for
02_retention_rfm	Repeat-purchase rate around 3.0%
03_delivery_reviews	Late median 2/5, on-time median 5/5
04_demand_forecast	Expected next-28-day volume around 6,660 orders

✖ **WARNING** The notebooks need the full real dataset. Run against tiny synthetic fixtures, some cells correctly fail — an empty group for the statistical test, or a date-range check expecting more coverage than 100 rows can provide. Always run 'make pipeline' with the real data first.

PART V — THE DASHBOARD

19. Power BI Desktop & DAX Guide

⚠️ **NOTE** The dashboard is built (dashboard/ecommerce_retention_dashboard_final.pbix). This section explains what it contains, how it connects to the database and how to refresh or rebuild it. There is no Power Query (M) transformation layer: every reshaping step happens in SQL, and Power BI imports finished tables.

✖ **WARNING** Power BI Desktop is Windows-only. On Mac or Linux, use Tableau Public with the same views and logic.

19.1 Install Power BI Desktop

If you do not have it installed already:

1. Open the Microsoft Store (search for it in the Start menu).
2. Search for 'Power BI Desktop'.
3. Click Get or Install. It is free — no account needed to install.
4. Wait for it to download and install (about 500 MB).
5. Open Power BI Desktop from the Start menu.

✔ **SUCCESS** When Power BI opens, you will see a splash screen. Close it (click the X in the top-right corner of the splash screen, not the app).

19.2 Connect to the Database

Opening the finished report needs no connection. Refreshing it does: the report imports from your local PostgreSQL database, so start the database first:

```
make db-up
```

Keep this terminal window open (or at least don't run make db-down). If you restart your computer, run this command again before opening the dashboard. Views must also be built (make views).

To create the connection from scratch, for example when rebuilding the report:

1. Home ribbon → Get data → More... → Database → PostgreSQL database → Connect.
2. Server: localhost:5432 · Database: olist · Data Connectivity mode: Import.
3. Credentials: choose Database, user analyst, password analyst (or your .env value), level set to olist, then Connect.

The PostgreSQL driver (Npgsql) ships bundled inside Power BI Desktop — nothing extra to install for a local database.

19.3 The Authentication Fix Already in Place

The first time this exact connection was attempted on this project, Power BI rejected valid credentials with a generic 'we couldn't authenticate' error. It was not a typo, because the identical credentials worked through a direct database connection outside Power BI. PostgreSQL 16 uses scram-sha-256 by default, and its documentation notes that this method is not supported by older client libraries. Switching the container to md5 authentication in docker-compose.yml fixed the failure, which is consistent with a client incompatibility with SCRAM; the driver version involved was not recorded. If your database was built from the current docker-compose.yml, this specific failure

should not recur. Because PostgreSQL no longer considers md5 secure, keep this setup to a local development machine.

> ✖ **WARNING** If you still see an SSL-related error (a separate, unrelated setting), open Advanced options on the connection dialog and set SSL mode to 'disable' — safe since the database is on your own machine.

19.4 Tables in the Final Model

The final report loads these seven objects from olist → analytics (do not import the raw staging tables):

- vw_sales
- vw_orders
- vw_customers
- vw_daily_orders
- forecast_28d
- dim_date
- order_payments

The report also contains cohort_retention and rfm_grid (read-only SELECTs of repository queries 12 and 15), forecast_chart (a calculated table that joins the last 90 days of actual orders to the 28 forecast rows) and _Measures (the 43 measures).

> ⚠ **NOTE** The Docker PostgreSQL container has a 64 MB /dev/shm, so parallel queries fail on refresh with error 53100. Every import therefore runs as a native query with the prefix SET max_parallel_workers_per_gather = 0; before an unchanged SELECT * FROM analytics.<name>, and parallel table loading is set to One. A permanent alternative is to add shm_size: 1gb to the PostgreSQL service in docker-compose.yml and recreate the container. The first refresh also shows a native-query approval dialog (choose Edit Permission, then Run).

19.5 Model Relationships

The final model has three many-to-one, single-direction relationships: dim_date[date] to the order_date column on vw_sales, vw_orders and vw_daily_orders. The other tables are deliberately unrelated, because they have no date, state or category key at a compatible grain, so the slicers do not affect them (payments, customers, cohorts, RFM and the forecast). When you build a model like this one, mark dim_date as the date table (required for the month-over-month measure).

> ✖ **WARNING** Always relate to order_date (a DATE column), never order_purchase_timestamp (a TIMESTAMP). Relating to the timestamp silently matches zero rows and leaves every chart blank — the single most common Power BI mistake on this project.

> ✖ **WARNING** On every monthly trend chart (revenue trend, AOV trend, late-% trend — Pages 1, 2, 4), put dim_date[date] on the X-axis and use Power BI's date hierarchy, not the standalone month integer column (1-12). This dataset spans two years (2016-2018); binning on the bare month number merges January 2017 and January 2018 into a single bucket and silently produces a meaningless trend line.

19.6 DAX Measures

The formulas below are the original design from dashboard/dax_measures.md, shown so that you can recreate the core measures. The built model's full list is in

dashboard/DAX_MEASURES_FINAL.md. To create a measure, right-click the named table → New measure → paste → Enter.

```
-- On vw_sales:
Total Revenue = SUM ( vw_sales[line_revenue] )
Total Orders = DISTINCTCOUNT ( vw_sales[order_id] )
AOV = DIVIDE ( [Total Revenue], [Total Orders] )

Revenue MoM % =
VAR CurrentRevenue = [Total Revenue]
VAR PriorRevenue =
    CALCULATE ( [Total Revenue], DATEADD ( dim_date[date], -1, MONTH ) )
RETURN
    DIVIDE ( CurrentRevenue - PriorRevenue, PriorRevenue )

-- On vw_orders:
Late Delivery % =
DIVIDE (
    CALCULATE ( COUNTROWS ( vw_orders ), vw_orders[is_late] = TRUE () ),
    COUNTROWS ( vw_orders )
)
Median Delivery Days = MEDIAN ( vw_orders[delivery_days] )
Avg Review Score = AVERAGE ( vw_orders[review_score] )
Pct 1-2 Star =
DIVIDE (
    CALCULATE ( COUNTROWS ( vw_orders ), vw_orders[review_score] IN { 1, 2 } ),
    CALCULATE ( COUNTROWS ( vw_orders ), NOT ISBLANK ( vw_orders[review_score] ) )
)

-- On vw_customers:
Repeat Cust % =
DIVIDE (
    CALCULATE (
        DISTINCTCOUNT ( vw_customers[customer_unique_id] ),
        vw_customers[is_repeat] = TRUE ()
    ),
    DISTINCTCOUNT ( vw_customers[customer_unique_id] )
)
```

The forecast needs no DAX: analytics.forecast_28d imports directly, with its forecast / lower / upper columns. The built report plots them with the last 90 days of actual orders from a calculated table, forecast_chart, which has no relationship to the slicers.

✔️ **SUCCESS** Sanity-check your model: Total Revenue ≈ 15,419,773.75 · Late Delivery % ≈ 8.1% · Repeat Cust % ≈ 3.0%. If yours differ substantially, recheck Section 19.5's relationships first.

The built model has 43 measures in the _Measures table, in five groups. Their full formulas are in dashboard/DAX_MEASURES_FINAL.md.

Group	Measures	What they cover
Revenue	12	Total Revenue, Total Orders, AOV, Revenue MoM %, Latest Month Revenue MoM %, category and seller rankings, Top Category Revenue, Category Cumulative Share %, Top 5 Category Share %, Top 10 Seller Revenue and Share %. Delivered orders.
Payments	4	Payment Records, Payment Record Share %, Avg Installments, Total Payment Value. Every payment record in any order status.
Customers & Retention	11	Customers, Repeat Customers, One-Time Customers, Repeat Cust %, Cohort Active Customers, Retention %, Month-1 Retention % (weighted), and the four RFM counts and shares. Delivered orders.
Operations & Satisfaction	9	Delivered Orders, Late Orders, Late Delivery %, Median Delivery Days, Avg and Median Review Score, Reviewed Orders, Pct 1-2 Star, Orders With Delivery Time. Delivered orders.
Forecast	7	Actual (last 90 d), Forecast, Lower 95%, Upper 95%, Forecast Total (28 days), Forecast Lower Sum, Forecast Upper Sum. All placed orders, and no slicer applies.

Three model objects are not measures. vw_orders[delivery_status] is a calculated column that returns Late, On time or 'Unknown (no delivery date)'; it tests for a blank is_late first, because in DAX BLANK() = FALSE() is TRUE and the first version counted eight undelivered-date orders as on-time. dim_date[month_start] is a calculated column used by the monthly axes and the rankings. forecast_chart is a calculated table. A measure recalculates for whatever the page is filtered to; a calculated column is computed once per row when the data is refreshed.

19.7 The Four Dashboard Pages

The original specification is in dashboard/BUILD_SPEC.md; small differences in the built pages are listed in dashboard/BUILD_QA_REPORT.md. Each page opens with the stakeholder question it answers.

Page 1 — Executive Overview: "Where is our revenue coming from, and is it growing?" (CFO)

- Slicers: order date range, customer state and product category.
- Four KPI cards: Total Revenue, Total Orders, Average Order Value and latest-month revenue growth.
- Monthly revenue and month-over-month growth (the growth axis is capped at −50% / +100% because the 2016 base is tiny).
- Category Pareto: the top 15 categories by revenue, with the cumulative share of all category revenue.
- Revenue by customer state: all 27 states, sorted by revenue.

Page 2 — Sales Deep-Dive: "What's driving sales, and how concentrated is it?" (CFO)

- Matrix of revenue by category (top 15) and month.
- Payment mix and installment behaviour. These count every payment record in any order status, so they do not reconcile to delivered revenue and ignore the slicers.
- Average order value by month.
- Top-10 sellers: their share of revenue, and a bar chart by revenue (anonymised seller IDs).

Page 3 — Customers & Retention: "Who returns, and which segments matter?" (CMO)

- KPI cards: unique customers (93,358), repeat customers (2,801), one-time customers (90,557) and the repeat rate (3.00%).
- Cohort heatmap: the share of each monthly cohort that bought again, by months since the first purchase, on a fixed 0-100% colour scale. It is a read-only import of repository query 12 (the cohort_retention table).
- One-time vs. repeat customers, month-1 retention (0.48%), and RFM segments by share of customers and by share of revenue.

Page 4 — Operations & Satisfaction: "How does delivery performance relate to customer satisfaction, and what order volume is expected next month?" (Ops)

- Slicers: order date range and customer state. They filter the delivery and review visuals but not the forecast.
- KPI cards: median delivery days (10.0), late delivery rate (8.11%), late delivered orders (7,826) and delivered orders (96,478).
- Delivery-time histogram and late-delivery % by purchase month.
- Review scores for late vs. on-time orders (average and median), the share of 1-2-star reviews, and a reviewed-orders table that lists the 8 orders with no delivery date as "Unknown". A text box states the Mann-Whitney result with its sample sizes.
- 28-day forecast chart (last 90 days of actuals, the forecast, and daily bounds), with cards for the forecast total (6,660) and the summed lower and upper bounds. A note explains that the summed bounds are not a 28-day interval.

19.8 Refresh, Save and Screenshots

1. To refresh: start the database (make db-up), open the report and choose Home → Refresh. Enter the database password from your .env file when asked.
2. If Power BI asks you to approve a native query, choose Edit Permission → Run.
3. Save with File → Save. The report is dashboard/ecommerce_retention_dashboard_final.pbix; it is the only current report file. Superseded copies are kept locally in dashboard/archive/ and are not in Git.
4. The page screenshots are in dashboard/screenshots/ (page1_executive_overview.jpg to page4_operations_satisfaction.jpg). If you retake them, keep the same names.

✔️ **SUCCESS** After a refresh the cards should read: Total Revenue R$15,419,773.75, Total Orders 96,478, late delivery rate 8.11%, repeat customer rate 3.00% and a forecast total of 6,660.

19.9 Known Limits

1. Static text: the Mann-Whitney statement on page 4 and the "roughly 97% buy once" note on page 3 are fixed text and do not update on refresh.
2. Accessibility is partial: alt text and reading order are set. The subtitles and footnotes that sit on the page background use #5B6472, which gives 5.48:1 against the 4.5:1 WCAG AA guideline (the earlier #6B7280 gave 4.43:1); screen-reader behaviour was not tested.
3. The payment visuals, the forecast and the customer, cohort and RFM visuals do not respond to the sales slicers. Each is labelled on its page.
4. The full list is in dashboard/BUILD_QA_REPORT.md.

PART VI — FINISHING THE PROJECT

20. The Insights Memo

The one-page executive memo is docs/insights_memo.md, with a PDF copy at pdf\INSIGHTS_MEMO.pdf. Every figure in it comes from the findings below or from the saved SQL outputs, so each sentence traces to a source.

20.1 Source Findings

docs/findings.md holds seven quantified findings with exact numbers and sources. The memo selects from these and from the saved SQL outputs:

#	Finding
1	Repeat-purchase rate is 3.0% (2,801 of 93,358 customers)
2	Late deliveries associate with significantly lower review scores (median 2/5 vs. 5/5, p < 0.001)
3	Late deliveries affect 8.11% of all delivered orders
4	Top 5 of 74 categories generate 39.25% of delivered revenue
5	Item-vs-payment totals reconcile within 0.30% of orders, and the gap is fully explained
6	RFM 'Champions' (1.29% of customers) generate 2.52% of revenue
7	The simple baseline forecast was not beaten by the more sophisticated model (WAPE 0.2308 vs. 0.2310)

20.2 Structure of the Memo

The memo fits on one A4 page. Every point follows one formula: number → what it means for the business → what to do. Its structure:

```
# Olist Marketplace: Revenue, Retention and Delivery

To: Leadership (CFO, CMO, Head of Operations)
Re: Annual planning - what the order data says, and what to do next
Data: Olist public dataset, 2016-2018; 99,441 orders, 96,478 delivered

## Business problem
Three leadership questions; every figure traces to docs/findings.md.

## Key findings (table: Finding | Evidence | What it means)
Six rows: repeat purchase, late delivery and reviews, uneven lateness,
category concentration, recent one-time buyers, forecast.

## Recommendations
Five actions: test a second-purchase offer, track on-time delivery,
check the cause of lateness, watch the top categories, and use the
forecast as a planning baseline.

## Limitations
Observational findings; one marketplace, about 20 months; no costs or
margins; retention and forecast scope limited.
```

- **Every number comes from docs/findings.md or from the saved SQL outputs.** Check any figure you plan to quote against its source before you quote it.
- **Written for a non-technical manager.** Short sentences, no jargon.

21. Before Your Final Push

⚠️ **NOTE** This section applies if you're pushing to YOUR OWN GitHub repo with your own credentials. If you're running this documentation just to verify the project works — a recruiter, a reviewer, or on a second machine without push access — stop after Section 20. Do not attempt to push to the original author's repository.

- If a Kaggle token was ever exposed (a screenshot, a shared screen), confirm it's been revoked and rotated.
- Add .ruff_cache/ to .gitignore — the linter's cache folder is not currently excluded there (only its contents self-exclude via its own internal rule).
- Do NOT run make clean as part of your push routine — it runs docker compose down -v FIRST, which destroys your entire database volume, not just caches. If you only want to clear Python/pytest caches: rm -rf .pytest_cache, then find . -type d -name __pycache__ -not -path './.venv/*' -exec rm -rf {} + (or just leave them — already gitignored, harmless either way).
- Confirm .env is not tracked (git status should never show it).
- Push to GitHub, then check the Actions tab, not just the README badge. It re-runs the pipeline and test suite against synthetic fixtures in a clean, GitHub-hosted PostgreSQL, independent of anything on your machine. A red badge means something doesn't reproduce outside your local environment and is worth investigating before treating the build as done.

PART VII — OPERATING THE PROJECT

22. Troubleshooting

Symptom	Cause	Fix
'docker' not recognized / 'failed to connect to the docker API'	Docker Desktop not installed or not running	Install/open Docker Desktop; wait for 'Engine running' before retrying
'make' not recognized	make not installed, or PATH not refreshed	Install via Scoop or Chocolatey (Section 7); open a brand-new terminal afterward
'kaggle' not recognized	Kaggle CLI's Scripts folder not on PATH	Run python --version to find your version, then: $env:Path = "...\AppData\Roaming\Python\Python<your version>\Scripts;$env:Path" (Section 8)
Kaggle 401 / authentication error	Missing or expired API token	kaggle.com → Settings → API → Create New Token; place at ~/.kaggle/kaggle.json
make db-up hangs, or 'port 5432 already allocated'	Another PostgreSQL (a local install, or a different container) is already using port 5432	Stop the other PostgreSQL, or change DB_PORT in .env to e.g. 5433 and re-run make db-up. Only the host-side port mapping changes — the container's internal port stays 5432
Container stuck on 'starting'	First-start initialization, or a bad prior shutdown	Wait 30 seconds; if it persists, make clean then make db-up
'Input validation: FAILED (N blocking failures ...)' — pipeline stops before building analytics	A raw CSV doesn't match the expected shape — a corrupted or partial download, or a different dataset version than this repo was built against	Read the blocking failures printed just above that line (each names the table/check); delete data/raw/ and re-run make pipeline to force a clean re-download before assuming the data itself is bad
Pipeline log shows ~100-row counts, not ~99,441	Ran against synthetic CI fixtures instead of real data	Delete data/raw/ and re-run make pipeline so it downloads the real dataset
'DependentObjectsStillExist' on make pipeline	An older schema was left behind from a prior run using non-CASCADE drops	Already fixed in this repo — every DROP TABLE IF EXISTS in sql/02_transform uses CASCADE (verified). If you see this error, you're on an old commit; git pull
Power BI: 'couldn't authenticate' with correct-looking credentials	Two distinct possible causes: (1) PostgreSQL 16's SCRAM auth vs. Power BI's driver — a real issue this project hit once; (2) .env's DB_PASSWORD was changed AFTER the container's data volume was already created, so the running database still has the old password	(1) Already fixed via md5 auth in docker-compose.yml (Section 9.2 / 19.3). (2) Either revert .env to match what the database actually has, or run docker compose down -v && make db-up && make pipeline to rebuild the database from scratch with the new password
Power BI: SSL connection error	Local Postgres not configured for SSL by default	Advanced options → SSL mode → disable (safe on localhost)
Power BI: every chart is blank	Date relationship built against the TIMESTAMP column	Rebuild using order_date (DATE), not order_purchase_timestamp
Notebooks fail with 'nonzero size' or similar errors	Run against tiny synthetic test data instead of real data	Run make pipeline against real Kaggle data first, then make notebooks

Symptom	Cause	Fix
ModuleNotFoundError: No module named 'src'	Ran jupyter or pytest directly instead of via make, or from the wrong working directory	Run make notebooks / make test, or manually launch with python -m ... from the repo root
Power BI refresh fails with PostgreSQL error 53100 (shared memory)	The Docker container's default 64 MB /dev/shm is too small for parallel queries	The final report already sets max_parallel_workers_per_gather = 0 in each query (Section 19.4). If you rebuild the report, add that prefix, or add shm_size: 1gb to docker-compose.yml and recreate the container

23. Best Practices & Maintenance

- Never hardcode credentials — everything reads from .env, and .env is never committed.
- Periodically re-run 'make clean && make db-up && make pipeline && make test' as a health check — a truly reproducible pipeline always lands on the same result.
- Update pinned dependency versions in requirements.txt deliberately, one at a time, re-running tests after each — never bulk-upgrade blindly.
- If a Kaggle token is ever exposed, revoke and regenerate it immediately.
- Add .ruff_cache/ to .gitignore (Section 21).
- Keep docs/insights_memo.md and pdf\INSIGHTS_MEMO.pdf consistent with docs/findings.md whenever a number changes.
- Rebuilding from scratch (make clean, then Sections 10-12 again) is the fastest way to confirm nothing has silently drifted.

24. Frequently Asked Questions

Question	Answer
Do I need to run 'make pipeline' every day?	No — once built, the database persists in its Docker volume. Only re-run to rebuild from scratch.
Why is the retention heatmap almost empty?	Because only 3.0% of customers return — that is the real finding, not a bug.
Why was the simple forecast model chosen?	It was just as accurate as the complex one (WAPE 0.2308 vs. 0.2310); the project honestly reports this rather than dressing up the fancier model as the winner.
Can I run this on Mac or Linux?	Yes for everything except the dashboard — Power BI Desktop is Windows-only; use Tableau Public instead with the same views.
Where are the dashboard screenshots?	They are in dashboard/screenshots/, one per page, taken from the final report.
Is the Kaggle dataset free to use for a portfolio?	It has historically carried a CC BY-NC-SA (non-commercial) license from its publisher — check the current Kaggle page before any commercial use; a personal portfolio is usually treated as non-commercial, but read the terms yourself.
What if my numbers don't match the findings exactly?	Small differences can occur if Kaggle's dataset snapshot has been updated. Investigate only if the difference is large — the population-level trends (3% repeat rate, the late-delivery effect) should hold regardless.

PART VIII — REFERENCE

25. Verifying Your Results

After running the real pipeline and notebooks, confirm your output matches the project's recorded findings (source: docs/findings.md):

#	Finding	Exact figures	Source
1	Repeat-purchase rate	3.0% — 2,801 of 93,358 customers	13_repeat_purchase_rate.sql
2	Late delivery vs. review score	Late: median 2/5, 54.07% 1-2★ (n=7,661). On-time: median 5/5, 9.22% 1-2★ (n=88,163). p < 0.001	notebooks/03_delivery_reviews.ipynb
3	Share of orders delivered late	8.11% — 7,826 of 96,478 delivered	09_late_pct_by_month.sql
4	Revenue concentration	Top 5 of 74 categories = 39.25% of revenue	03_category_pareto.sql
5	Payment reconciliation	303 of 99,441 orders (0.30%) differ by >R$0.01; fully explained	16_items_vs_payments_reconciliation.sql
6	RFM Champions	1.29% of customers generate 2.52% of revenue	14_rfm_customer_scores.sql, 15_rfm_segment_share.sql
7	28-day forecast	6,660 orders; daily bounds sum to 3,585-9,943 (not a 28-day interval); baseline not beaten by Holt-Winters (WAPE 0.2308 vs. 0.2310)	notebooks/04_demand_forecast.ipynb

25.1 Independent Re-run, October 2026

The project was rebuilt from the raw CSV files in a throwaway PostgreSQL 16.2 database (Linux, Python 3.10.12) outside the repository. The full record is in dashboard/BUILD_QA_REPORT.md.

Check	Result
Pipeline, 10 tests, 18 SQL outputs, dashboard views	Pass. 0 blocking failures; 10 passed; every saved output identical once line endings are ignored; view row counts as in Section 11.
Notebooks 01 to 04	Pass. Source cells unchanged; printed outputs identical apart from timestamps.
Excel summary	Pass with a note. Pivot sheets identical; the Raw Extract sheet is a first-5,000-row sample (Section 11).
Final .pbix in Power BI Desktop	Pass in the pre-delivery audit. Not repeated since: the file's hash is unchanged.
Not performed	A Power BI refresh against the Docker database, Docker and make on Windows, the Kaggle download, and GitHub Actions CI.

26. Final Verification Checklist

☐ docker --version, python --version, git --version, make --version all succeed

☐ Project cloned; .env created with the 5 DB_ lines

- ☐ docker compose ps shows olist_postgres as 'healthy' (after make db-up)
- ☐ make setup completed with no ERROR line
- ☐ make pipeline ended with 'Pipeline complete' against the real dataset
- ☐ make views ended with 'Dashboard views built'
- ☐ make test shows '10 passed'
- ☐ make notebooks completed for all 4 notebooks with no errors
- ☐ Notebook 03 shows median 2/5 (late) vs. 5/5 (on-time)
- ☐ Power BI refreshes the report without errors (database container running)
- ☐ Dashboard relationships use order_date, not the timestamp column
- ☐ KPI cards match reference values (Total Revenue ≈ 15.4M, Late % ≈ 8.1%, Repeat % ≈ 3.0%)
- ☐ The final .pbix and the page screenshots are in dashboard/ (already in the repository)
- ☐ docs/insights_memo.md checked against the findings in Section 25
- ☐ .ruff_cache/ added to .gitignore; make clean run before the final push
- ☐ Pushed to GitHub; CI badge is green

27. Glossary

Term	Plain-English meaning
Pipeline	The automated sequence turning raw data into the finished warehouse
Schema	A labelled area inside a database grouping related tables (stg and analytics)
Staging	The raw, untouched copy of source data, before cleaning
Star schema	A layout with fact tables in the middle and dimension tables around them
Fact table	A table of events or measurements (e.g. orders, order items)
Dimension table	A table of descriptive details (e.g. customers, products, dates)
Container	An isolated mini-environment (via Docker) that runs a program without installing it directly
Volume	Docker's way of saving a container's data so it survives restarts
View	A saved query that behaves like a table; used to feed the dashboard
Primary / foreign key	A column that uniquely identifies a row / links one table to another
DAX	The formula language used inside Power BI to create measures
Measure	A Power BI calculation that updates as you filter the dashboard
Cohort	A group of customers who first bought in the same month, tracked over time
RFM	Recency, Frequency, Monetary — a way of scoring customers by buying behaviour
WAPE	Weighted Absolute Percentage Error — how forecast accuracy is measured
Walk-forward	A forecasting test that only ever trains on past data to predict future data
Idempotent	Safe to run repeatedly — running it twice gives the same result as once
CI	Continuous Integration — automated checks run on every code push

28. Appendix: Project at a Glance

Attribute	Value
GitHub	https://github.com/MARCUS-00/ecommerce-retention-analytics
Dataset	Brazilian E-Commerce Public Dataset by Olist (Kaggle)
Dataset size	~100,000 orders, 9 tables, 2016-2018
Python version	3.12
Database	PostgreSQL 16 (via Docker)
Python modules	8 (in src/)
SQL files	32 (2 schema, 8 transform, 18 analysis, 4 views)
Notebooks	4
Automated tests	10
Setup time (first run)	~30 minutes
Cost	₹0 — all tools are free

28.1 Quick Command Reference

Command	What it does
`make setup`	Install all Python packages (first time only)
`make db-up / make db-down`	Start / stop the database container
`make pipeline`	Download data and build the full analytics warehouse
`make views`	Build the Power BI feeding views
`make excel`	Build the Excel summary file
`make test`	Run all 10 data quality tests
`make notebooks`	Execute all 4 analysis notebooks, including the forecast
`make clean`	Destroy the database and reset everything